메모리 관리 최적화

개요

vLLM의 가장 혁신적인 기술 중 하나는 PagedAttention을 통한 메모리 관리 최적화입니다. 기존의 LLM 추론 시스템들이 메모리 단편화와 비효율적인 KV 캐시 관리로 인해 성능 병목을 겪었다면, vLLM은 운영체제의 가상 메모리 개념을 차용한 블록 기반 메모리 관리를 통해 이러한 문제를 해결했습니다.

이 문서에서는 vLLM의 메모리 관리 시스템의 핵심 구성 요소들을 코드 레벨에서 분석하고, 각 최적화 기법이 어떻게 성능 향상을 달성하는지 상세히 살펴보겠습니다.

기존 방식의 한계점

전통적인 KV 캐시 관리의 문제점

기존의 Transformer 기반 LLM 추론에서는 다음과 같은 메모리 관리 문제들이 있었습니다:

1. **연속적 메모리 할당의 비효율성**

- 각 시퀀스마다 최대 길이만큼 연속된 메모리를 미리 할당
- 실제 시퀀스 길이가 짧을 경우 메모리 낭비 발생
- 메모리 단편화로 인한 활용률 저하

2. **정적 배치 크기의 한계**

- 고정된 배치 크기로 인한 GPU 활용률 저하
- 시퀀스 길이 차이로 인한 패딩 오버헤드
- 메모리 부족 시 배치 크기 감소 필요

3. **KV 캐시 복사 오버헤드**

- 빔 서치나 샘플링 시 KV 캐시 전체 복사 필요
- 메모리 대역폭 낭비와 지연시간 증가

```
# 기존 방식의 비효율적인 메모리 할당 예시
class TraditionalKVCache:
def  __init__(self, max_seq_len, batch_size, num_heads, head_dim):
# 최대 길이만큼 미리 할당 (메모리 낭비)
self.k_cache = torch.zeros(batch_size, num_heads, max_seq_len, head_dim)
self.v_cache = torch.zeros(batch_size, num_heads, max_seq_len, head_dim)
self.seq_lens = [0] * batch_size
def  append_kv(self, batch_idx, k, v):
seq_len = self.seq_lens[batch_idx]
# 연속적 메모리에 저장 (단편화 발생 가능)
self.k_cache[batch_idx, :, seq_len] = k
self.v_cache[batch_idx, :, seq_len] = v
self.seq_lens[batch_idx] += 1
```## PagedAttention: vLLM의 혁신적 해결책

핵심 아이디어

PagedAttention은 운영체제의 가상 메모리 시스템에서 영감을 받아 KV 캐시를 고정 크기의 블록

1. **메모리 단편화 해결**: 블록 단위 할당으로 메모리 활용률 극대화

2. **동적 메모리 관리**: 필요에 따라 블록을 할당/해제

3. **효율적인 복사**: 블록 포인터만 복사하여 오버헤드 최소화

4. **메모리 공유**: 동일한 프리픽스를 가진 시퀀스 간 블록 공유

블록 기반 메모리 구조

```python
# vLLM의 블록 기반 KV 캐시 구조
class Block:
def __init__(self, block_id: int, block_size: int):
self.block_id = block_id
self.block_size = block_size # 예: 16 토큰
self.token_ids = []
self.ref_count = 1  # 참조 카운트 (공유를 위해)
def is_full(self) -> bool:
return len(self.token_ids) == self.block_size
def append_token_ids(self, token_ids: List[int]):
self.token_ids.extend(token_ids)
```

SelfAttnBlockSpaceManager: 핵심 메모리 관리자

클래스 구조 및 역할

`SelfAttnBlockSpaceManager` 는 vLLM의 메모리 관리 시스템의 핵심 클래스로, KV 캐시의 할당, 해제, 스와핑을 담당합니다.

```
class SelfAttnBlockSpaceManager(BlockSpaceManager):
"""BlockSpaceManager which manages the allocation of KV cache.
주요 기능:
- KV 캐시 블록 할당 및 해제
- 메모리 스와핑 (GPU ↔ CPU)
- 슬라이딩 윈도우 메모리 최적화
- 프리픽스 캐싱을 통한 메모리 공유
"""
def __init__(
self,
block_size: int, # 블록당 토큰 수 (예: 16)
num_gpu_blocks: int, # GPU 블록 수
num_cpu_blocks: int, # CPU 블록 수
watermark: float = 0.01, # 메모리 워터마크
sliding_window: Optional[int] = None, # 슬라이딩 윈도우 크기
enable_caching: bool = False, # 프리픽스 캐싱 활성화
) -> None:
```### 초기화 과정 분석


```python
def __init__(self, ...):
```

```
self.block_size = block_size

self.num_total_gpu_blocks = num_gpu_blocks

self.num_total_cpu_blocks = num_cpu_blocks

# 슬라이딩 윈도우 설정

self.sliding_window = sliding_window

self.max_block_sliding_window = None

if sliding_window is not None:

# 슬라이딩 윈도우에 필요한 최대 블록 수 계산

num_blocks = sliding_window // block_size + 1

# 마지막 블록이 부분적으로 채워질 수 있으므로 +1

self.max_block_sliding_window = num_blocks + 1

# 워터마크 기반 메모리 관리

self.watermark = watermark

self.watermark_blocks = int(watermark * num_gpu_blocks)

# 블록 할당자 생성

self.block_allocator = CpuGpuBlockAllocator.create(

allocator_type="prefix_caching"  if enable_caching else  "naive",

num_gpu_blocks=num_gpu_blocks,

num_cpu_blocks=num_cpu_blocks,

block_size=block_size,

)

# 블록 테이블 관리
```

```
self.block_tables: Dict[SeqId, BlockTable] = {}
```

메모리 할당 가능성 검사

```
def can_allocate(self, seq_group: SequenceGroup, num_lookahead_slots: int = 0)
"""시퀀스 그룹에 대한 메모리 할당 가능성을 검사합니다."""
seq = seq_group.get_seqs(status=SequenceStatus.WAITING)[0]
# 필요한 블록 수 계산
num_required_blocks = BlockTable.get_num_required_blocks(
seq.get_token_ids(),
block_size=self.block_size,
num_lookahead_slots=num_lookahead_slots,
)
# 슬라이딩 윈도우 적용
if self.max_block_sliding_window is not None:
num_required_blocks = min(num_required_blocks, self.max_block_sliding_window)
num_free_gpu_blocks = self.block_allocator.get_num_free_blocks(Device.GPU)
# 워터마크 기반 할당 결정
if (self.num_total_gpu_blocks - num_required_blocks < self.watermark_blocks):
return AllocStatus.NEVER # 절대 할당 불가
if num_free_gpu_blocks - num_required_blocks >= self.watermark_blocks:
return AllocStatus.OK # 즉시 할당 가능
else:
```

```
return AllocStatus.LATER # 나중에 할당 가능
```

```## BlockTable: 시퀀스별 블록 관리

### BlockTable의 역할과 구조

`BlockTable`은 개별 시퀀스의 토큰들을 블록 단위로 관리하는 클래스입니다.

```python
class BlockTable:
"""특정 시퀀스의 블록들을 관리하는 클래스
주요 기능:
- 토큰 시퀀스를 블록으로 매핑
- 블록 할당 및 해제
- 동적 블록 확장
- 슬라이딩 윈도우 지원
"""
def __init__(
self,
block_size: int,
block_allocator: DeviceAwareBlockAllocator,
_blocks: Optional[List[Block]] = None,
max_block_sliding_window: Optional[int] = None,
):
```
```

```
self._block_size = block_size

self._allocator = block_allocator

self._blocks: BlockList = BlockList(_blocks or [])

self._max_block_sliding_window = max_block_sliding_window

self._num_full_slots = self._get_num_token_ids()
```

블록 할당 과정

```
def  allocate(self, token_ids: List[int], device: Device = Device.GPU,

extra_hash: Optional[int] = None) -> None:

"""토큰 ID 시퀀스에 대해 블록을 할당합니다."""

assert not self._is_allocated

assert token_ids

# 토큰 ID들을 위한 블록 할당

blocks = self._allocate_blocks_for_token_ids(

prev_block=None,

token_ids=token_ids,

device=device,

extra_hash=extra_hash

)

self.update(blocks)

self._num_full_slots = len(token_ids)

```python
```
```

```
def _allocate_blocks_for_token_ids(self, prev_block: Optional[Block],
token_ids: List[int], device: Device,
extra_hash: Optional[int] = None) -> List[Block]:
"""토큰 ID들을 블록 크기로 나누어 할당합니다."""
blocks: List[Block] = []
block_token_ids = []
tail_token_ids = []
# 토큰들을 블록 크기로 청킹
for cur_token_ids in chunk_list(token_ids, self._block_size):
if len(cur_token_ids) == self._block_size:
block_token_ids.append(cur_token_ids) # 완전한 블록
else:
tail_token_ids.append(cur_token_ids) # 부분 블록
# 완전한 블록들 할당 (불변 블록)
if block_token_ids:
blocks.extend(
self._allocator.allocate_immutable_blocks(
prev_block, block_token_ids=block_token_ids,
device=device, extra_hash=extra_hash
)
)
prev_block = blocks[-1]
# 부분 블록 할당 (가변 블록)
```

```
if tail_token_ids:
assert len(tail_token_ids) == 1
cur_token_ids = tail_token_ids[0]
block = self._allocator.allocate_mutable_block(
prev_block=prev_block, device=device, extra_hash=extra_hash
)
block.append_token_ids(cur_token_ids)
blocks.append(block)
return blocks
```

동적 토큰 추가

```
def append_token_ids(self, token_ids: List[int], num_lookahead_slots: int = 0,
num_computed_slots: Optional[int] = None,
extra_hash: Optional[int] = None) -> None:
"""기존 블록에 새로운 토큰들을 추가합니다."""
assert self._is_allocated, "no blocks have been allocated"
assert len(self._blocks) > 0
``` # 슬라이딩 윈도우로 인한 블록 드롭
if self._max_block_sliding_window is not None:
null_block = self._allocator.allocate_or_get_null_block()
assert num_computed_slots is not None
더 이상 필요하지 않은 블록들을 null 블록으로 교체
```
```

```
end_block_idx = (num_computed_slots // self._block_size) - self._max_block_slidi

for idx in  range(0, end_block_idx):

b = self._blocks[idx]

if b is not null_block:

self._allocator.free(b)

self._blocks[idx] = null_block

# 새로운 토큰과 lookahead 슬롯을 위한 공간 확보

self.ensure_num_empty_slots(

num_empty_slots=len(token_ids) + num_lookahead_slots,

extra_hash=extra_hash

)

# 블록에 토큰 추가

first_block_idx = self._num_full_slots // self._block_size

token_blocks = self._chunk_token_blocks_for_append(token_ids)

for i, token_block in  enumerate(token_blocks):

self._blocks.append_token_ids(first_block_idx + i, token_block)

self._num_full_slots += len(token_ids)
```

CpuGpuBlockAllocator: 디바이스 간 블록 관리

이중 메모리 풀 구조

`CpuGpuBlockAllocator` 는 CPU와 GPU 메모리 풀을 동시에 관리하여 메모리 스와핑을 지원합니다.

```
class CpuGpuBlockAllocator(DeviceAwareBlockAllocator):

"""CPU와 GPU 메모리에서 블록을 할당할 수 있는 블록 할당자

주요 기능:

- CPU/GPU 메모리 풀 분리 관리

- 디바이스 간 블록 스와핑

- 프리픽스 캐싱 지원

- Copy-on-Write 최적화

"""

``` def  __init__(self, cpu_block_allocator: BlockAllocator,

gpu_block_allocator: BlockAllocator):

# CPU와 GPU 할당자 분리

self._allocators = {

Device.CPU: cpu_block_allocator,

Device.GPU: gpu_block_allocator,

}

# 스와핑 매핑 추적

self._swap_mapping: Dict[int, int] = {}

self._null_block: Optional[Block] = None

# 블록 ID to 할당자 매핑

self._block_ids_to_allocator: Dict[int, BlockAllocator] = {}

for _, allocator in  self._allocators.items():

for block_id in allocator.all_block_ids:
```
```

```
self._block_ids_to_allocator[block_id] = allocator
```

블록 할당 메서드

```
def allocate_mutable_block(self, prev_block: Optional[Block], device: Device,
extra_hash: Optional[int] = None) -> Block:
"""지정된 디바이스에 가변 블록을 할당합니다."""
return self._allocators[device].allocate_mutable_block(
prev_block, extra_hash=extra_hash
)

def allocate_immutable_blocks(self, prev_block: Optional[Block],
block_token_ids: List[List[int]], device: Device,
extra_hash: Optional[int] = None) -> List[Block]:
"""지정된 디바이스에 불변 블록들을 할당합니다."""
return self._allocators[device].allocate_immutable_blocks(
prev_block, block_token_ids, extra_hash=extra_hash
)

def free(self, block: Block) -> None:
"""블록이 차지한 메모리를 해제합니다."""
if isinstance(block, NullBlock):
return # Null 블록은 해제하지 않음
```

```
block_id = block.block_id

assert block_id is not None

allocator = self._block_ids_to_allocator[block_id]

allocator.free(block)

```## 메모리 스와핑과 워터마크 기반 관리


스와핑 메커니즘

vLLM은 GPU 메모리가 부족할 때 블록을 CPU 메모리로 스와핑하여 더 많은 시퀀스를 처리할 수 있


```python

def  swap_out(self, seq_group: SequenceGroup) -> List[Tuple[int, int]]:

"""시퀀스 그룹의 블록들을 GPU에서 CPU로 스와핑합니다."""

physical_block_id_mapping = []

for seq in seq_group.get_seqs(status=SequenceStatus.RUNNING):

blocks = self.block_tables[seq.seq_id].blocks

if  len(blocks) == 0:

continue

# 블록 스와핑 실행

seq_swap_mapping = self.block_allocator.swap(

blocks=blocks,

src_device=Device.GPU,

dst_device=Device.CPU
```

```
)

# 블록 테이블 업데이트

self.block_tables[seq.seq_id].update(blocks)

# 물리적 블록 ID 매핑 생성

seq_physical_block_id_mapping = {

self.block_allocator.get_physical_block_id(Device.GPU, gpu_block_id):

self.block_allocator.get_physical_block_id(Device.CPU, cpu_block_id)

for gpu_block_id, cpu_block_id in seq_swap_mapping.items()

}

physical_block_id_mapping.extend(

list(seq_physical_block_id_mapping.items())

)

return physical_block_id_mapping

```### 워터마크 기반 메모리 관리



```python

def  can_allocate(self, seq_group: SequenceGroup, num_lookahead_slots: int = 0)

"""워터마크를 고려한 할당 가능성 검사"""

# 필요한 블록 수 계산

num_required_blocks = self._calculate_required_blocks(seq_group, num_lookahead_s

num_free_gpu_blocks = self.block_allocator.get_num_free_blocks(Device.GPU)

# 워터마크 기반 결정

if (self.num_total_gpu_blocks - num_required_blocks < self.watermark_blocks):
```

```
return AllocStatus.NEVER # 워터마크 이하로 떨어질 경우

if num_free_gpu_blocks - num_required_blocks >= self.watermark_blocks:

return AllocStatus.OK # 워터마크 이상 유지 가능

else:

return AllocStatus.LATER # 추후 할당 가능
```

슬라이딩 윈도우 메모리 최적화

슬라이딩 윈도우의 개념

슬라이딩 윈도우는 매우 긴 시퀀스에서 최근 토큰들만 유지하여 메모리 사용량을 제한하는 기법입니다.

```
def  append_token_ids(self, token_ids: List[int], ...):

"""슬라이딩 윈도우를 고려한 토큰 추가"""

# 슬라이딩 윈도우로 인한 오래된 블록 제거

if  self._max_block_sliding_window is not None:

null_block = self._allocator.allocate_or_get_null_block()

# 계산된 토큰 수를 기반으로 제거할 블록 범위 결정

end_block_idx = (num_computed_slots // self._block_size) - self._max_block_slidi

for idx in  range(0, end_block_idx):

b = self._blocks[idx]

if b is not null_block:

self._allocator.free(b) # 메모리 해제
```

```
self._blocks[idx] = null_block # null 블록으로 교체
```### NullBlock을 통한 메모리 절약

```python
class NullBlock(Block):
"""슬라이딩 윈도우로 인해 드롭된 KV 캐시 블록의 플레이스홀더
실제 블록을 래핑하여 수정을 방지하고, isinstance() 검사를 통해
null 블록 여부를 확인할 수 있게 합니다.
"""
def __init__(self, proxy: Block):
super().__init__()
self._proxy = proxy
def append_token_ids(self, token_ids: List[BlockId]):
raise ValueError("null block should not be modified")
@property
def block_id(self):
return self._proxy.block_id
# 기타 프로퍼티들은 프록시를 통해 접근하되 수정은 차단
```

성능 향상 효과 분석

메모리 활용률 개선

PagedAttention을 통한 블록 기반 메모리 관리는 다음과 같은 성능 향상을 달성합니다:

1. **메모리 활용률 향상**

- 기존 방식: 60-70% (패딩으로 인한 낭비)
- PagedAttention: 90-95% (블록 단위 할당)

2. **메모리 단편화 해결**

- 고정 크기 블록으로 단편화 최소화
- 동적 할당/해제로 메모리 재사용 극대화

3. **배치 크기 증가**

- 효율적인 메모리 사용으로 더 큰 배치 처리 가능
- GPU 활용률 향상으로 처리량 증대

```python

# 성능 비교 예시 (가상의 벤치마크 결과)

class PerformanceComparison:

"""메모리 관리 방식별 성능 비교"""

traditional_approach = {

"memory_utilization": 0.65, # 65% 활용률

"max_batch_size": 32, # 최대 배치 크기

"throughput_tokens_per_sec": 1200, # 처리량

"memory_overhead": 0.35, # 35% 오버헤드

}

paged_attention = {

"memory_utilization": 0.92, # 92% 활용률
```

```
"max_batch_size": 64, # 2배 증가

"throughput_tokens_per_sec": 2400, # 2배 증가

"memory_overhead": 0.08, # 8% 오버헤드

}

@staticmethod

def calculate_improvement():

return {

"memory_efficiency": 0.92 / 0.65, # 1.42배 향상

"throughput_improvement": 2400 / 1200, # 2배 향상

"batch_size_increase": 64 / 32, # 2배 증가

}
```

Copy-on-Write 최적화

```
def  fork(self, parent_seq: Sequence, child_seq: Sequence) -> None:

"""Copy-on-Write를 통한 효율적인 시퀀스 포킹"""

if parent_seq.seq_id not in self.block_tables:

return

src_block_table = self.block_tables[parent_seq.seq_id]

# 실제 복사 없이 블록 테이블만 포크

self.block_tables[child_seq.seq_id] = src_block_table.fork()

# 자식 시퀀스 추적 시작

self._last_access_blocks_tracker.add_seq(child_seq.seq_id)
```

```python
def clear_copy_on_writes(self) -> List[Tuple[int, int]]:
"""Copy-on-Write 상태를 정리하고 매핑을 반환"""
device = Device.GPU
return self._allocators[device].clear_copy_on_writes()
```## 프리픽스 캐싱을 통한 추가 최적화

공통 프리픽스 공유

동일한 프리픽스를 가진 시퀀스들 간에 블록을 공유하여 메모리 사용량을 더욱 줄입니다:

```python
def get_common_computed_block_ids(self, seqs: List[Sequence]) -> List[int]:
"""공통으로 계산된 블록 ID들을 찾아 프리필 스킵 가능한 블록 결정"""
computed_seq_block_ids = []
for seq in seqs:
all_blocks = self.block_tables[seq.seq_id].physical_block_ids
num_cached_tokens = self._computed_blocks_tracker.get_num_cached_tokens(seq)
# 캐시된 토큰 수를 블록 수로 변환
assert num_cached_tokens % self.block_size == 0
num_cached_blocks = num_cached_tokens // self.block_size
computed_block_ids = all_blocks[:num_cached_blocks]
computed_seq_block_ids.append(computed_block_ids)
```

```
# 공통 계산된 블록 ID 반환
return  self.block_allocator.get_common_computed_block_ids(computed_seq_block_id
```

CUDA 커널 레벨 최적화

PagedAttention CUDA 구현

vLLM의 PagedAttention은 CUDA 커널 레벨에서 최적화되어 있습니다:

```
// csrc/attention/paged_attention_v1.cu에서 발췌
template <typename T, typename CACHE_T, int BLOCK_SIZE,
vllm::Fp8KVCacheDataType KV_DTYPE, bool IS_BLOCK_SPARSE,
int NUM_THREADS = 128>
void paged_attention_v1_launcher(
torch::Tensor& out, torch::Tensor& query, torch::Tensor& key_cache,
torch::Tensor& value_cache, int num_kv_heads, float scale,
torch::Tensor& block_tables, torch::Tensor& seq_lens, int max_seq_len,
const std::optional<torch::Tensor>& alibi_slopes, torch::Tensor& k_scale,
torch::Tensor& v_scale, const int tp_rank,
const int blocksparse_local_blocks, const int blocksparse_vert_stride,
const int blocksparse_block_size, const int blocksparse_head_sliding_step) {
``` // 블록 테이블을 통한 간접 메모리 접근
int* block_tables_ptr = block_tables.data_ptr<int>();
// 각 헤드와 시퀀스에 대해 병렬 처리
```
```

```
dim3 grid(num_heads, num_seqs, 1);

dim3 block(NUM_THREADS);

// 헤드 크기에 따른 최적화된 커널 선택

switch (head_size) {

case 64:

LAUNCH_PAGED_ATTENTION_V1(64);

break;

case 128:

LAUNCH_PAGED_ATTENTION_V1(128);

break;

// ... 기타 헤드 크기들

}

}
```

결론

vLLM의 PagedAttention 기반 메모리 관리 시스템은 다음과 같은 핵심 혁신을 통해 LLM 추론의 성능을 획기적으로 향상시켰습니다:

주요 혁신 사항

1. **블록 기반 메모리 관리**: 운영체제의 가상 메모리 개념을 차용하여 메모리 단편화 해결
2. **동적 메모리 할당**: 필요에 따른 블록 할당/해제로 메모리 활용률 극대화
3. **효율적인 스와핑**: CPU-GPU 간 블록 단위 스와핑으로 더 많은 시퀀스 처리
4. **슬라이딩 윈도우**: 긴 시퀀스에서 메모리 사용량 제한

5. **Copy-on-Write**: 블록 공유를 통한 복사 오버헤드 최소화

6. **프리픽스 캐싱**: 공통 프리픽스 공유로 메모리 효율성 극대화

성능 향상 결과

- **메모리 활용률**: 65% → 92% (1.42배 향상)
- **처리량**: 2-24배 증가 (모델과 워크로드에 따라)
- **배치 크기**: 2배 이상 증가 가능
- **지연시간**: 메모리 관리 오버헤드 최소화로 감소

```
PagedAttention은 단순한 메모리 최적화를 넘어서 LLM 서빙 패러다임을 바꾼 혁신적 기술입니다

이러한 메모리 관리 최적화는 vLLM의 다른 최적화 기법들(배치 처리, 병렬 처리, 커널 최적화)과

### 향후 발전 방향

- **적응형 블록 크기**: 워크로드에 따른 동적 블록 크기 조정
- **계층적 메모리 관리**: SSD 등 추가 저장 계층 활용
- **예측적 프리페칭**: AI 기반 메모리 접근 패턴 예측
- **하드웨어 특화 최적화**: 새로운 GPU 아키텍처에 맞춘 최적화

vLLM의 메모리 관리 시스템은 LLM 추론 서빙 분야에서 새로운 표준을 제시했으며, 향후 더욱 발전
```